

쇼핑 쇼츠 자동화 시스템 MVP 설계

A. 전체 시스템 개요

이 시스템의 목적은 “시청자가 15~30초 만에 특정 상품이 자신의 상황에 맞는지 빠르게 판단하게 돕는 상황 해결형 쇼츠 채널”을 만드는 것입니다. 즉, 단순히 제품을 나열하지 않고, **상품 사용 맥락에 맞춘 쇼츠 콘텐츠**를 자동 또는 반자동으로 생성해 수익형 제휴 링크로 연결합니다. 초기 니치는 데스크·모바일·생산성 액세서리로 한정하여, 전문성 있는 카테고리에서 빠르게 시행착오를 거칩니다.

시스템에서는 **Antigravity**가 전체 워크플로를 오케스트레이션하며 예약 작업과 에이전트 관리를 담당합니다. **Claude Code**(대규모 AI 코딩 에이전트)는 Antigravity 노드 내부에서 코드 작성, 파일 생성·수정, 자동화 로직 구현 등 개발 업무를 수행합니다. 예를 들어, 특정 노드의 작업 로직을 구현하거나 스키마 파일을 생성하는 작업에 Claude Code를 활용합니다.

운영은 **완전 자동화**가 아니라 **반자동 승인 구조**로 설계해야 합니다. 이는 **규제 및 플랫폼 정책** 준수, 콘텐츠 과장·오용 방지, 품질 확보 등을 위해 인간 검수가 필요하기 때문입니다. 예를 들어, 제휴 링크 고지나 부작용이 의심되는 표현은 사람이 반드시 검수해야 합니다. 실제로 TikTok 등의 플랫폼은 제휴 광고 공개 설정을 켜지 않으면 게시물을 제거할 수 있으므로 【17†L348-L353】, 최종 업로드 전 사람이 승인하여 문제를 예방합니다.

데이터 흐름은 아래와 같습니다. 먼저 **상품 후보 → 상품 정제 → 상품 스코어링 → 승인 → 장면 브리프 → 대본 생성 → 미디어 프롬프트 생성 → 영상 생성 큐 → 미디어 검수 → 업로드 패키지 준비 → 업로드 → 성과 수집 → 다음 후보 추천** 순으로 진행됩니다. 이를 통해 **분석 결과가 곧 다음 콘텐츠 기획**으로 반영됩니다. 최신 쇼츠 분석 툴들은 단순 분석을 넘어 “무엇을 만들어야 할지 추천”하는 수준까지 발전하고 있어 【11†L108-L111】, 성과 데이터와 트렌드 인사이트를 활용해 다음 상품을 선택합니다.

B. 전체 워크플로 다이어그램 (텍스트 설명)

각 단계는 Antigravity 노드로 구성되며, 흐름은 다음과 같습니다:

- 상품 후보 수집 노드 → 상품 정제 노드 → 상품 스코어링 노드 → 승인 대기 노드 →
- 규제/표현 리스크 검사 노드 → 제휴 링크 감사 노드 → 경쟁 쇼츠 패턴 분류 노드 → 장면 브리프 생성 노드 → 대본 생성 노드 → 이미지/영상 프롬프트 생성 노드 → 영상 생성 작업 큐 노드 →
- 미디어 결과 검수 노드 → 자막/CTA/고지 문구 생성 노드 → 업로드 패키지 생성 노드 → 수동 승인 노드 → 업로드 수행 → 성과 로그 수집 노드 → 다음 실험 추천 노드.

각 화살표는 작업 완료 시 자동 또는 예약으로 다음 노드를 호출하는 순차적 흐름을 나타냅니다. 실패나 예외가 발생하면 해당 단계에서 복구하거나, 필요시 이전 단계(예: 상품 정제 재시도, 다른 상품 대안 탐색)로 되돌아갑니다. 예를 들어, 상품 스코어링 중 필요한 정보가 누락되면 상품 정제 단계로 돌아가 추가 데이터를 요청할 수 있습니다. 최종 업로드 전 **수동 승인 노드**가 있어 사람이 최종 콘텐츠를 검토하며, 그 사이 중대한 오류가 발견되면 해당 노드 이전의 워크플로로 되돌릴 수 있습니다.

C. Antigravity 노드 설계표

각 노드를 아래 표로 정리합니다. 주요 노드명을 예로 들었으며, 요구 사항에 따라 추가/삭제 가능합니다.

노드명	목적	트리거 방식	입력 데이터	처리 내용	출력 데이터	Claude Code 작업 구현	사람 승인 필요 지점	실패 처리	우선 순위
상품 후보 수집	신규 상품 목록 수집	스케줄링 (주기적)	없음 혹은 미리 정의된 상품 키워드	제휴사 API(예: Coupang 등) 호출 또는 웹 스크래핑을 통해 상품 목록 수집	상품 메타데이터 목록 (ID, 이름, URL 등)	상품 수집 코드 (API 호출, 결과 파싱) 생성	없음 (자동)	실패 시 재 시도하거나 백업 데이터 사용	높음
상품 정제	수집된 상품 필터링 및 분류	상품 수집 완료 트리거	상품 메타데이터 목록	중복 제거, 카테고리 할당, 설명 및 이미지 미존재 상품 제거, 기본 정보 보완	정제된 상품 리스트	중복 제거/데이터 정제 코드 작성	없음 (자동)	문제 상품 필터링 후 로그 기록	높음
상품 스코어링	상품 우선 순위 부여	상품 정제 완료 트리거	정제된 상품 리스트	상품 리뷰 요약 (GPT 활용), 상품군 특징 반영 스코어 계산, 기대 성과 예측	스코어링된 상품 리스트	리뷰 요약 및 스코어링 로직 구현 (MCP 활용 가능)	최종 상품 승인	낮은 스코어 상품 제외	중

노드명	목적	트리거 방식	입력 데이터	처리 내용	출력 데이터	Claude Code 작업 구현	사람 승인 필요 지점	실패 처리	우선 순위
규제/표현 리스크 검사	법적/광고 규제 위반 여부 검증	상품 스코어 링 완료 트리거	스코어 링된 상품 리스트	건강/의료/화장품 관련 과장 표현 점검, 약 속성 표현 (FDA/FTC 가이드 참고), 부적절 키워드 필터링 【19 + L429-L437】 【22 + L105-L113】	위험 여부 표시 (OK/위험) 상품 리스트	규제 가이드라인 기반 체크 코드 작성	위험 상품 리스트 검토	위험 표시 상품 제외	높음
제휴 링크 감사	유효한 제휴 링크 확인 및 표시 검토	규제 검사 통과 트리거	OK 표시 상품 리스트	각 상품에 대한 제휴 링크 생성 혹은 검증, 필요한 경우 새 링크 생성, 문구 포함 검사	제휴 링크 포함 상품 리스트	제휴 링크 생성/검증 로직 구현	제휴 문구 검토	링크 누락 시 알림, 백업 링크 사용	중

노드명	목적	트리거 방식	입력 데이터	처리 내용	출력 데이터	Claude Code 작업 구현	사람 승인 필요 지점	실패처리	우선순위
경쟁 쇼츠 패턴 분류	경쟁사 쇼츠 콘텐츠 유형 및 트렌드 분석	상품 확장 트리거	승인된 상품 리스트 (1개)	쇼츠 플랫폼 검색(키워드/카테고리) 후 유사 상품 쇼츠 조회, 스타일(스크립트 형식, 후킹 방식 등) 분류 【11 + L90-L98】	경쟁 쇼츠 패턴 메타 정보	영상 패턴 분석 로직 구현 (유사도 계산 등)	없음 (자동)	데이터 부족 시 샘플 확대	중
장면 브리프 생성	쇼츠 구성용 시각적/스토리 브리프 작성	경쟁 패턴 분석 후 자동	상품 상세 정보, 경쟁 패턴	제품 특징과 패턴 기반으로 쇼츠 각 장면별 상황/캡션 브리프 생성	장면별 브리프 목록	GPT를 활용한 장면 설명 생성	없음 (자동)	생성실 패시 패턴 단순화	중

노드명	목적	트리거 방식	입력 데이터	처리 내용	출력 데이터	Claude Code 작업 구현	사람 승인 필요 지점	실패 처리	우선 순위
쇼츠 대본 생성	쇼츠 내용 성/텍스트 대사 및 내레이션 생성	장면 브리프 생성 트리거	장면 브리프	장면별 상황 설명, 상품 특징점 강조 대본 작성 (Hook, 문제 제기, 해결 제안, CTA 포함)	쇼츠 대본 스크립트	GPT를 이용한 스크립트 생성	없음 (자동)	길이/ 톤 검토 후 재 생 성	중
이미지/영상 프롬프트 생성	시 미 디 어 생 성 용 프 로 프 트 작 성	대 본 생 성 완 료 트 리 거	대 본 스 크 립 트, 장 면 브 리 프	각 장면 설명에 맞춰 이미지/영상 생성 프롬프트(텍스트) 생성, 스타일/구도/색감 지시 포함	프롬프트 목록 (scene1_prompt, scene2_prompt...)	GPT를 활용해 텍스트 프롬프트 생성	없음 (자동)	부 적 절 프 로 프 트 교 정	중

노드명	목적	트리거 방식	입력 데이터	처리 내용	출력 데이터	Claude Code 작업 구현	사람 승인 필요 지점	실패 처리	우선 순위
Runway/Synthesia API 큐 노드	AI 미디어 (영상/이미지) 생성 요청 및 큐 관리	프롬프트 생성 완료 트리거	이미지/영상 프롬프트 목록	Runway/Synthesia/Stability API 호출로 영상/음성/이미지 생성, 요청/응답을 대기열로 관리 【23 + L58-L60】 【26 + L947-L954】	생성된 미디어 파일 (또는 링크)	API 연동 코드 작성 (비동기 큐 관리)	없음 (자동)	API 호출 실패 시 재시도	높음
미디어 결과 검수 노드	생성된 영상/이미지 품질 및 정책 검수	영상 생성 완료 트리거	생성된 미디어 파일	생성물 품질 점검(해상도/노이즈/오류), 상품·정책 위반 표현 검사, 불량 시 재생성 요청	검수 통과 파일, 이슈 파일 표시	결과 파일 검증 로직 작성 (예: 모션 블러 체크)	미디어 검수	재요청은 프롬프트 수정	높음

노드명	목적	트리거 방식	입력 데이터	처리 내용	출력 데이터	Claude Code 작업 구현	사람 승인 필요 지점	실패 처리	우선 순위
자막/CTA/고지 문구 생성	영상 자막, 콜투액션, 제휴 고지 문구 생성	미디어 검수 통과 트리거	대본 스크립트	최종 대본 기반 자막 텍스트 생성, CTA 문구 및 제휴 링크 고지(예: “#광고, 제휴 링크 포함”) 추가	자막 파일(.srt 등), CTA/고지 텍스트	GPT로 자막 및 문구 작성	고지 문구 검수	오타/미흡 수정 후 재생성	중
업로드 패키지 생성 노드	최종 영상 조합 및 업로드 준비	자막 생성 완료 트리거	검수된 미디어 파일, 자막 파일	영상, 자막, 썸네일 등을 조합하여 최종 업로드 가능한 패키지 생성. 비디오 포맷/길이 등 검증	최종 업로드 파일(영상+자막+설명)	파일 결합 스크립트, 메타데이터 작성 코드 구현	없음 (자동)	오류 시 패키지 재생성	중

노드명	목적	트리거 방식	입력 데이터	처리 내용	출력 데이터	Claude Code 작업 구현	사람 승인 필요 지점	실패 처리	우선 순위
수동 승인 노드	사람이 최종 콘텐츠를 검토하고 승인	업로드 패키지 준비 완료 트리거	최종 업로드 파일 패키지	사람이 최종 영상 및 설명을 검토하여 업로드 승인 또는 거부. 정책/표현 이상 유무 확인	승인/거부 여부, 수정 요청 사항	지침 문서 생성 (사용자 인터페이스 안내 등)	최종 업로드 승인	거부 시 수정 → 재검토	높음
성과 로그 수집 노드	업로드 후 성과 데이터 수집 및 정리	업로드 후 타이머 트리거	채널 영상 ID	YouTube API 등으로 조회수·클릭률·조회시간 등의 성과 지표 수집 및 저장	성과 데이터 (CSV/DB)	API 연동 및 데이터베이스 저장 로직 작성	없음 (자동)	수집 실패 시 재시도	중

노드명	목적	트리거 방식	입력 데이터	처리 내용	출력 데이터	Claude Code 작업 구현	사람 승인 필요 지점	실패 처리	우선 순위
다음 실험 추천 노드	성과 분석 결과물을 기반으로 다음 상품 제안	성과 수집 완료 트리거	성과 데이터, 경쟁 분석 자료	수집된 성과와 과거 결과를 분석하여 개선 사항 도출. GPT로 유망 카테고리/상품 추천	추천 상품 리스트	GPT로 분석 후 추천 로직 구현	없음 (자동)	데이터 부족 시 휴면	낮음

참고: 쇼츠 업로드 전 반드시 “이 콘텐츠는 광고를 포함합니다” 등의 제휴 고지 문구를 포함해야 합니다 【17 + L369-L372】 【17 + L399-L402】. 예를 들어 TikTok에서는 제휴광고 설정을 켜지 않으면 게시물이 삭제될 수 있으므로 【17 + L348-L353】, 시스템에서 자동으로 해시태그(#광고, #affiliatelink 등)를 생성해 사람이 검토하도록 합니다. 또한 건강·미용 제품의 경우 **강한 효능 주장**은 규제 위험이 크므로 사전에 필터링합니다 【22 + L105-L113】 【19 + L429-L437】.

D. 각 노드별 상세 구현 설명

아래는 주요 노드별 상세 내용입니다. 각 노드는 Antigravity에서 독립적으로 동작하며, 필요한 파일 구조와 입력/출력을 정의해 Claude Code가 구현하게 됩니다.

상품 후보 수집 노드

- **역할:** 제휴사 상품 API 또는 웹스크래핑을 통해 신규 상품 후보를 주기적으로 수집합니다. 예: “데스크 액세서리” 키워드로 상품 목록을 가져옴.
- **Antigravity 구성:** 스케줄 타이머(예: 매일/매주 실행)로 트리거되며, 완료 시 다음 노드로 이어집니다.
- **Claude Code 작업:** `/nodes/product_candidate_collection/` 폴더를 생성하여 수집 로직 구현. 예를 들어 `collect_products.py` 에서 API 호출 코드를 작성하고, `input_schema.json`, `output_schema.json`, 샘플 입력/출력 JSON을 작성합니다.
- **파일 구조:**

```
/nodes/product_candidate_collection/
- product_candidate_collection.md (노드 설명)
```

- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- collect_products.py

- **입력 예시:** (없거나) `"keywords": ["무선키보드", "마우스패드"]`
- **출력 예시:** `[{"id":123, "name":"XYZ 키보드", "category":"keyboard", "url":"..."}, ...]`
- **검증 기준:** 출력된 상품 리스트가 비어있지 않고, 필수 필드(id,name,url)가 있는지 검사합니다.
- **실패 처리:** API 호출 오류나 빈 결과면 재시도하거나, 예외 로깅 후 휴면 상태로 전환하여 운영자 개입 대기.
- **다음 노드 데이터:** 상품 메타데이터 리스트를 **상품 정제 노드**로 넘깁니다.

상품 정제 노드

- **역할:** 수집된 상품 중 데이터 누락, 중복, 카테고리 오류 등을 정제합니다. 불량 상품(이미지 없음, 상세정보 부족 등)을 필터링하여 품질을 높입니다.
- **Antigravity 구성:** 상품 수집 완료 데이터가 들어오면 자동 트리거됩니다.
- **Claude Code 작업:** `/nodes/product_refinement/` 폴더에서 중복 제거와 누락 보완 로직 구현. `refine_products.py` 를 작성하여 받아온 리스트를 정제하고, 카테고리 재분류를 시도합니다.
- **파일 구조:**

```
/nodes/product_refinement/
- product_refinement.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- refine_products.py
```

- **입력 예시:** `[{"id":123, "name":"XYZ 키보드", "url":"...", "images":["..."]}, ...]`
- **출력 예시:** `[{"id":123, "name":"XYZ 키보드", "url":"...", "images":["..."], "category":"electronics"}, ...]`
- **검증 기준:** 출력된 상품 리스트에서 누락 필드(id,url)가 없는지, 중복 id가 제거되었는지 확인합니다.
- **실패 처리:** 필수 정보 부족 상품은 로그에 남기고 제외합니다. 예: 이미지가 없으면 해당 상품은 다음 단계에서 제외.
- **다음 노드 데이터:** 정제된 상품 리스트를 **상품 스코어링 노드**로 전달합니다.

상품 스코어링 노드

- **역할:** 각 상품의 **판매 가능성 또는 콘텐츠 적합도**를 평가하여 우선순위를 매깁니다. 예: 상품 리뷰 내용을 요약해 긍정/부정 점수를 계산하거나, 검색량 트렌드 등을 반영합니다.
- **Antigravity 구성:** 상품 정제 완료 데이터를 받으면 실행됩니다.
- **Claude Code 작업:** `/nodes/product_scoring/` 폴더에서 상품 리뷰 요약 및 점수 계산 로직을 구현합니다. 예를 들어 `score_products.py` 에서 GPT를 호출하여 리뷰를 요약한 뒤, 가중치를 적용해 스코어링합니다.
- **파일 구조:**

```

/nodes/product_scoring/
- product_scoring.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- score_products.py

```

- **입력 예시:** [{"id":123,"name":"XYZ 키보드","reviews":[...],"score":null}, ...]
- **출력 예시:** [{"id":123,"name":"XYZ 키보드","score":4.5}, ...]
- **검증 기준:** 스코어 값이 지정 범위(예: 0~5점) 내에 있는지, 모든 상품에 스코어가 할당되었는지 확인합니다.
- **실패 처리:** 스코어 산출 실패 시 기본값(예: 중간점수) 부여 후 계속 진행하거나, 해당 상품을 일단 낮은 순위로 처리합니다.
- **다음 노드 데이터:** 스코어링된 리스트를 **규제/표현 리스크 검사 노드**로 넘깁니다. 사람 승인은 여기서 최종 상품 선별 단계에 포함될 수 있습니다.

규제/표현 리스크 검사 노드

- **역할:** 건강·의료·화장품 등 민감 카테고리에서 과장·허위 과장 광고 요소가 없는지 검사합니다. 예를 들어, “주름 제거” 같은 표현은 FDA에서 의약품으로 보는 약속성 표현이므로 사전에 걸러냅니다 【22†L105-L113】. 또한 FTC 가이드에 따라 “건강 효과” 주장은 과학적 근거가 필요함을 감안합니다 【19†L429-L437】.
- **Antigravity 구성:** 상품 스코어링 후 자동 트리거됩니다.
- **Claude Code 작업:** /nodes/regulation_check/ 폴더에서 규제 검사 로직을 작성합니다. check_regulations.py 에서 상품 설명이나 키워드를 분석하여 금지어(예: “치료”, “완벽”, “즉시 효과” 등)를 탐지합니다.
- **파일 구조:**

```

/nodes/regulation_check/
- regulation_check.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- check_regulations.py

```

- **입력 예시:** [{"id":123,"name":"XYZ 화장품","description":"주름 완화 효과"} , ...]
- **출력 예시:** [{"id":123,"risk":true,"reason":"주름 완화 표현은 FDA 규제 대상"} , ...]
- **검증 기준:** 리스크 검사 결과에 따라 risk: true/false 가 올바르게 표시됐는지, 위험 사유(금지 표현)가 명시됐는지 확인합니다.
- **실패 처리:** 로직 오류 시 기본적으로 risk:false 로 표시하고 다음 단계로 이동하되, 반드시 로그를 기록하여 운영자가 검토할 수 있게 합니다.
- **다음 노드 데이터:** risk:false 인 상품만 **제휴 링크 감사 노드**로 넘깁니다. (사람 검토: 리스크 상품 목록을 관리자에게 보내 재검토를 요청할 수 있습니다.)

제휴 링크 감사 노드

- **역할:** 각 상품별로 유효한 제휴 링크를 생성하거나 기존 링크를 확인하고, 제휴 광고 고지 문구가 포함됐는지 검사합니다.

- **Antigravity 구성:** 규제 검사를 통과한 상품이 들어오면 실행됩니다.
- **Claude Code 작업:** `/nodes/affiliate_link_audit/` 폴더에서 제휴 링크 생성 및 검증 로직을 작성합니다. 예를 들어 `audit_affiliate_links.py` 를 통해 제휴 프로그램 API를 호출해 링크를 만들고, 링크 설명에 “이 포스트는 제휴 링크를 포함합니다” 등의 문구를 자동 생성합니다.
- **파일 구조:**

```
/nodes/affiliate_link_audit/
- affiliate_link_audit.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- audit_affiliate_links.py
```

- **입력 예시:** `[{"id":123,"name":"XYZ 마우스","url":"..."} , ...]`
- **출력 예시:** `[{"id":123,"affiliate_url":"https://...","disclosure":"#광고"} , ...]`
- **검증 기준:** 모든 상품에 대해 `affiliate_url` 이 생성되었는지, `disclosure` 필드에 적절한 고지문구 (e.g., `#광고` 등)가 포함되었는지 확인합니다.
- **실패 처리:** 제휴 API 오류 시 재시도하고, 그래도 실패하면 해당 상품을 일시적으로 제외합니다.
- **다음 노드 데이터:** 링크가 확인된 상품 정보를 **경쟁 쇼츠 패턴 분류 노드**로 전달합니다. 사람 검토: 생성된 고지문구와 링크를 사람이 한번 검토하여 오타나 정책 위반이 없는지 확인합니다.

경쟁 쇼츠 패턴 분류 노드

- **역할:** 선정된 상품과 유사한 쇼츠 콘텐츠(경쟁자 쇼츠)를 수집하여 포맷·스타일을 분석합니다. 예를 들어, 경쟁사 쇼츠에서 자주 쓰이는 후킹 방식(문제 제시→해결), 카메라 구도 패턴 등을 분류합니다 【11 + L90-L98】 .
- **Antigravity 구성:** 제휴 링크가 포함된 상품 하나가 들어오면 트리거됩니다.
- **Claude Code 작업:** `/nodes/competitor_pattern/` 폴더에서 경쟁사 데이터 수집·분석 코드를 만듭니다. 유튜브/틱톡 API로 인기 쇼츠를 조회한 뒤, `classify_patterns.py` 로 스타일을 카테고리화합니다.
- **파일 구조:**

```
/nodes/competitor_pattern/
- competitor_pattern.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- classify_patterns.py
```

- **입력 예시:** `{"product_id":123,"keywords":["무선 마우스","키보드"]}`
- **출력 예시:** `{"patterns":["바로 사용 데모", "제품 특징 열거", "가격 비교", "CTA 삽입"]}`
- **검증 기준:** 반환된 패턴 리스트가 적절한지(빈 리스트 아님) 확인합니다.
- **실패 처리:** 데이터 부족시 범용 패턴(예: ‘제품 리뷰’)으로 채우거나, 이 단계를 생략합니다.
- **다음 노드 데이터:** 분석된 패턴 정보를 **장면 브리프 생성 노드**로 넘깁니다.

장면 브리프 생성 노드

- **역할:** 경쟁 패턴과 상품 특징을 종합해 쇼츠의 각 장면별 상황이나 구도를 브리핑합니다. 예: ① 문제 상황 제시, ② 제품 사용 모습, ③ 결과 강조 등.
- **Antigravity 구성:** 경쟁 패턴 분석 완료 시 실행됩니다.
- **Claude Code 작업:** `/nodes/scene_brief/` 폴더에서 `create_scene_brief.py` 를 작성해 GPT가 장면별 설명을 하도록 지시합니다. 복수의 장면을 구조화된 리스트로 출력하도록 스키마를 정의합니다.
- **파일 구조:**

```
/nodes/scene_brief/  
- scene_brief.md  
- input_schema.json  
- output_schema.json  
- sample_input.json  
- sample_output.json  
- create_scene_brief.py
```

- **입력 예시:** `{"product_id":123,"patterns":["오버뷰","사용방법"], "features":["경량","Bluetooth"]}`
- **출력 예시:**
`["장면1: 책상 위에서 케이스를 열어 무선 마우스를 보여줌.", "장면2: 사용자가 마우스를 빠르게 움직이며 사용 방법 설명.", "장면3: 마우스 배터리 교체 모습을 보여줌."]`
- **검증 기준:** 각 장면 설명에 상품 정보가 반영됐는지, 모호한 표현은 없는지 확인합니다.
- **실패 처리:** GPT 응답이 불완전하면 시스템에서 프롬프트를 조정하여 재요청합니다.
- **다음 노드 데이터:** 생성된 장면 리스트를 쇼츠 대본 생성 노드로 전달합니다.

쇼츠 대본 생성 노드

- **역할:** 장면 브리프를 바탕으로 실제 영상 내레이션 또는 텍스트 대본을 생성합니다. Hook, 문제, 해결, CTA 순서로 구성합니다.
- **Antigravity 구성:** 장면 브리프가 완성되면 트리거됩니다.
- **Claude Code 작업:** `/nodes/script_generation/` 폴더에 `generate_script.py` 를 만들어, GPT를 이용해 스토리텔링 대본을 작성하도록 합니다. 스크립트의 길이와 톤을 사전에 설정합니다.
- **파일 구조:**

```
/nodes/script_generation/  
- script_generation.md  
- input_schema.json  
- output_schema.json  
- sample_input.json  
- sample_output.json  
- generate_script.py
```

- **입력 예시:** `["장면1: ...", "장면2: ..."]`
- **출력 예시:**
`["첫 장면에서는 바쁜 책상 위에 ...라고 말한다.", "두 번째 장면에서는 무선 연결 방법을 설명하며 ...라고 말한다."]`
- **검증 기준:** 대본이 15~30초 분량에 맞는지, 상품 이름과 핵심 기능이 포함되었는지 체크합니다.

- **실패 처리:** 대본이 너무 길거나 짧으면, GPT에 길이 제한을 재조정하여 재생성합니다.
- **다음 노드 데이터:** 생성된 대본 텍스트를 **이미지/영상 프롬프트 생성 노드**로 보냅니다.

이미지/영상 프롬프트 생성 노드

- **역할:** 각 장면의 시각적 연출을 위해 AI 이미지/영상 생성용 텍스트 프롬프트를 만듭니다. 배경, 카메라 앵글, 감정 등 스타일 요소를 포함합니다.
- **Antigravity 구성:** 스크립트 생성 완료 시 자동 트리거됩니다.
- **Claude Code 작업:** `/nodes/media_prompt/` 폴더에서 `create_media_prompts.py`를 구현해, GPT가 시각적 설명 기반의 프롬프트를 생성하도록 합니다. 예를 들어, “책상 위에 흑백 현대식 마우스 사진” 같은 구체적 설명을 만듭니다.
- **파일 구조:**

```
/nodes/media_prompt/
- media_prompt.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- create_media_prompts.py
```

- **입력 예시:** `{"script":["첫 장면: ...말한다"]}`
- **출력 예시:** `{"prompts":["busy desk setup with modern wireless mouse and keyboard, overhead angle, bright lighting", "close-up of hand connecting bluetooth on mouse, focused shot", ...]}`
- **검증 기준:** 각 프롬프트에 제품과 관련된 키워드(예: 제품명, 색상)가 포함됐는지 확인합니다.
- **실패 처리:** 부정확한 프롬프트가 생성되면 GPT 재요청 또는 키워드 보강을 수행합니다.
- **다음 노드 데이터:** 완성된 프롬프트 리스트를 **Runway/Synthesis API 작업 큐 노드**로 전송합니다.

Runway/Synthesis/Stability API 작업 큐 노드

【45 + embed_image】 촬영용 클랩보드 이미지를 예시로, 이 노드는 실제 **AI 미디어 생성 API**를 호출하여 영상을 생성합니다. Runway API(비디오 생성), Synthesis API(아바타 영상), Stability API(이미지) 등의 엔드포인트를 사용합니다.

- **역할:** 미리 생성된 프롬프트를 기반으로 AI 서비스로 미디어 생성을 요청하고, 병렬 작업 큐로 관리합니다. Runway의 Gen-4 모델, Synthesis의 아바타 생성 등 최신 모델을 활용하여 빠르게 결과를 얻습니다. 【23 + L58-L60】 【26 + L947-L954】 .
- **Antigravity 구성:** 프롬프트 생성 완료 시 트리거되며, API 호출과 응답을 비동기로 처리합니다.
- **Claude Code 작업:** `/nodes/media_generation_queue/` 폴더에서 `media_queue.py`와 어댑터 모듈을 만듭니다. 큐에 들어온 각 프롬프트를 비디오/이미지로 변환하고 결과 파일을 저장하거나 S3 등에 업로드합니다. 예제 스크립트와 키 관리 코드를 작성합니다.
- **파일 구조:**

```
/nodes/media_generation_queue/
- media_queue.md
- input_schema.json
- output_schema.json
- sample_input.json
```

```
- sample_output.json
- media_queue.py
- runway_adapter.py
- synthesia_adapter.py
- stability_adapter.py
```

- **입력 예시:** `{"prompts":["desk with mouse","user holding phone"]}`
- **출력 예시:** `{"videos":["scene1.mp4"],"images":["scene2.png"]}`
- **검증 기준:** 반환된 파일 형식과 길이를 검사(예: 영상은 15~30초). 파일이 손상 없이 저장됐는지 확인합니다.
- **실패 처리:** API 호출 실패 시 적절한 리트라이 루틴을 구현합니다. 실패가 반복되면 로그를 남기고 해당 장면을 재프롬프트하도록 표시합니다.
- **다음 노드 데이터:** 생성된 미디어 파일 경로를 **미디어 결과 검수 노드**로 전달합니다.

미디어 결과 검수 노드

- **역할:** 생성된 영상/이미지가 품질 기준을 충족하고 정책 위반 요소가 없는지 검수합니다. 해상도, 시각적 오류, 제품 노출 적절성 등을 확인합니다.
- **Antigravity 구성:** 미디어 생성이 완료되면 트리거됩니다.
- **Claude Code 작업:** `/nodes/media_inspection/` 폴더에서 `inspect_media.py` 를 작성해 간단한 검수 로직(예: 블러 감지, 모델 오브젝트 포함 검사 등)을 구현합니다. 필요 시 사람이 볼 수 있게 썸네일을 생성합니다.
- **파일 구조:**

```
/nodes/media_inspection/
- media_inspection.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- inspect_media.py
```

- **입력 예시:** `{"videos":["scene1.mp4"],"images":["scene2.png"]}`
- **출력 예시:** `{"videos":["scene1.mp4"],"status":"passed","issues":[]}`
- **검증 기준:** 동영상의 프레임 수, 이미지 해상도가 기준에 부합하는지, 콘텐츠가 흐릿하지 않은지 확인합니다.
- **실패 처리:** 미디어에 중대한 문제가 있으면 해당 장면을 다시 생성하도록 마커를 붙입니다. 예: 화질 불량시 재생성 요청.
- **다음 노드 데이터:** 통과한 미디어 파일들을 **자막/CTA/고지 문구 생성 노드**로 보냅니다. (사람 검수: 생성 영상과 텍스트 대본을 사람이 최종 확인하여 어색한 부분이 없는지 검토합니다.)

자막/CTA/고지 문구 생성 노드

- **역할:** 대본을 바탕으로 영상의 **자막**과 화면 텍스트(CTA, 제휴 고지 등)를 생성합니다. 예: “지금 바로 링크를 확인하세요” 같은 콜투액션과 “#광고” 고지 등을 자막에 포함합니다.
- **Antigravity 구성:** 미디어가 통과하면 트리거됩니다.
- **Claude Code 작업:** `/nodes/subtitle_cta/` 폴더에서 `generate_subtitles.py` 와 `generate_cta.py` 를 만듭니다. GPT로 대본을 시간대별 자막으로 변환하고, CTA/고지 텍스트를 별도 생성합니다.
- **파일 구조:**

```

/nodes/subtitle_cta/
- subtitle_cta.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- generate_subtitles.py
- generate_cta.py

```

- **입력 예시:** `{"script":["첫 장면: ...","두 번째 장면: ..."]}`
- **출력 예시:** `{"subtitles":["00:00:01,000 --> 00:00:03,000\n(나레이션) 제품 XYZ는...", "..."], "cta":["상세보기 (제휴 링크 포함)","disclosure":"#광고"]}`
- **검증 기준:** 자막 싱크가 올바르고, CTA/고지 문구가 노출 문장에 적절히 포함됐는지 확인합니다.
- **실패 처리:** 생성된 자막에 나타나 타임코드 오류 시 수정합니다. CTA/고지가 누락되면 자동으로 `#광고` 태그를 추가합니다.
- **다음 노드 데이터:** 자막 파일(SRT)과 CTA/고지 문구를 **업로드 패키지 생성 노드**로 전달합니다.

업로드 패키지 생성 노드

- **역할:** 영상, 자막, 썸네일, 설명 텍스트 등 업로드에 필요한 모든 요소를 조합하여 최종 포맷을 만듭니다.
- **Antigravity 구성:** 자막/CTA 문구 생성 완료 시 트리거됩니다.
- **Claude Code 작업:** `/nodes/upload_package/` 폴더에서 `create_upload_package.py`를 작성하여, 영상과 자막을 하나의 포맷으로 인코딩하고, 동영상 설명(제목, 본문, 태그)에 제품 정보 및 제휴 링크를 추가합니다.
- **파일 구조:**

```

/nodes/upload_package/
- upload_package.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- create_upload_package.py

```

- **입력 예시:** `{"video":"final_video.mp4","subtitles":"subtitles.srt","cta":["상품보기","disclosure":"#광고"]}`
- **출력 예시:** `{"package_path":"upload_package_123.zip","description":"이 동영상은 XYZ 제휴 링크를 포함합니다. #광고", ...}`
- **검증 기준:** 패키지가 완성된 후 업로드 규격(해상도, 길이, 파일 포맷)과 설명에 제휴 링크 및 고지가 포함됐는지 확인합니다.
- **실패 처리:** 패키지 생성 오류 시 자동 재생성하고, 문제가 지속되면 관리자 알림을 보냅니다.
- **다음 노드 데이터:** 완성된 업로드 패키지는 **수동 승인 노드**로 넘어갑니다.

수동 승인 노드

- **역할:** 사람이 최종 결과물을 검토하여 채널 업로드 여부를 결정합니다. 정책 위반, 표현 과장 등이 없는지 최종 확인합니다.

- **Antigravity 구성:** 업로드 패키지가 준비되면 대기 상태로 들어가며, 관리자가 승인을 하면 다음 단계(업로드)로 진행됩니다.
- **Claude Code 작업:** `/nodes/manual_approval/` 폴더에 `approval_workflow.md`를 작성하여 승인 절차를 문서화합니다. UI 알림이나 승인 API 연동 코드가 필요한 경우 구현 지시를 포함시킵니다.
- **파일 구조:**

```
/nodes/manual_approval/
- manual_approval.md
- approval_checklist.md
```

- **입력 예시:** 관리자가 검토할 수 있도록 “검토용 링크”(예: 임시 스트리밍 링크)가 제공됩니다.
- **출력 예시:** `{"approved":true,"notes":""}`
- **검증 기준:** 관리자가 승인 버튼을 클릭했는지, 설명에 문제점 체크리스트(정책 위반, 과장 등)가 없는지 기록합니다.
- **실패 처리:** 거부 시 거부 사유를 기록하고 해당 노드부터 재수정 작업을 진행합니다.
- **다음 노드 데이터:** 승인된 패키지는 **업로드 수행** 단계로 넘어갑니다. (이후 업로드는 Antigravity 외부(YouTube API 등)에서 처리)

성과 로그 수집 노드

- **역할:** 업로드된 쇼츠의 조회수, 클릭률(CTR), 시청 시간 등 성과 지표를 주기적으로 수집하여 저장합니다.
- **Antigravity 구성:** 업로드 후 일정 시간이 지나면 자동으로 트리거됩니다. 예: 업로드 24시간 후, 7일 후 등.
- **Claude Code 작업:** `/nodes/performance_logging/` 폴더에서 `collect_performance.py`를 작성하여 YouTube Data API를 호출, 영상 ID로 조회수를 가져와 DB에 저장합니다.
- **파일 구조:**

```
/nodes/performance_logging/
- performance_logging.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- collect_performance.py
```

- **입력 예시:** `{"video_id":"abcd1234","since":"2026-06-15T00:00:00Z"}`
- **출력 예시:** `{"views":1500,"likes":50,"click_rate":0.045}`
- **검증 기준:** 수집된 데이터가 예상치 범위 내에 있는지(예: 갑자기 0% CTR 등 비정상치 검출) 확인합니다.
- **실패 처리:** API 호출 실패 시 재시도하고, 데이터가 심각히 부족하면 로그에 표시하여 추후 확인합니다.
- **다음 노드 데이터:** 성과 데이터가 **다음 실험 추천 노드**로 전달됩니다.

다음 실험 추천 노드

- **역할:** 축적된 성과와 트렌드 데이터를 바탕으로 **다음에 실험할 상품/콘텐츠 방향**을 추천합니다. 예를 들어, 특정 상품군에서 높은 전환이 나왔다면 유사 상품을, 시청 시간이 짧았다면 쇼츠 패턴 수정을 제안합니다
[11†L108-L111] .
- **Antigravity 구성:** 최신 성과 로그가 수집되면 트리거됩니다.
- **Claude Code 작업:** `/nodes/next_experiment/` 폴더에서 `recommend_next.py`를 작성하여 GPT가 이전 성과, 경쟁 패턴, 사용자 피드백을 종합해 다음 아이디어를 생성하도록 합니다.
- **파일 구조:**

```

/nodes/next_experiment/
- next_experiment.md
- input_schema.json
- output_schema.json
- sample_input.json
- sample_output.json
- recommend_next.py

```

- **입력 예시:** {"last_video_id": "abcd1234", "performance": {"views": 1500, "ctr": 0.045}}
- **출력 예시:** {"recommendations": ["기존 마우스 대신 새로운 키보드 소개 콘텐츠", "영상 길이를 20초로 줄여보기"]}
- **검증 기준:** 추천 아이디어의 타당성을 사람 검토로 확인하고, 주요 피드백(예: 시청자가 원하는 정보 포함 여부)을 기록합니다.
- **실패 처리:** 유효한 추천을 못 찾으면 기본 템플릿(예: 베스트셀러 상품 검토)으로 돌려보거나, 더 많은 데이터를 기다립니다.
- **다음 노드 데이터:** 추천 내용을 기반으로 실제 다음 상품 후보 수집 주기에 반영됩니다.

E. Claude Code 구현 지시안

아래는 각 노드를 구현하기 위해 Claude Code에게 줄 수 있는 작업 예시입니다. 각 노드 폴더 생성, 문서화, 스키마·샘플 데이터 준비, 코드 목업 등을 포함합니다.

Claude Code 작업 지시:

- /nodes/product_candidate_collection/ 폴더를 생성한다.
- product_candidate_collection.md에 노드 목적, 트리거, 입력/출력, 실패 처리를 문서화한다.
- input_schema.json과 output_schema.json을 작성한다.
- sample_input.json과 sample_output.json을 만든다.
- collect_products.py에 제휴사 상품 API 호출 및 응답 파싱 코드를 작성한다 (mock 데이터로 테스트).
- /nodes/product_refinement/ 폴더를 생성한다.
- product_refinement.md를 문서화하고, refine_products.py에서 중복 제거 및 필터링 로직을 구현한다.
- /nodes/product_scoring/ 폴더를 생성한다.
- product_scoring.md에 상품 스코어링 목적과 방식을 정리하고 score_products.py를 작성한다 (예: 리뷰 요약 및 점수 계산).
- /nodes/regulation_check/ 폴더를 생성한다.
- regulation_check.md를 작성하고, check_regulations.py에서 금지 키워드 검사 로직을 구현한다 (규제 가이드 참고).
- /nodes/affiliate_link_audit/ 폴더를 생성한다.
- affiliate_link_audit.md를 작성하고, audit_affiliate_links.py에서 제휴 링크 생성/검증 로직을 구현한다 (디스클로저 포함).
- /nodes/competitor_pattern/ 폴더를 생성한다.
- competitor_pattern.md를 작성하고, classify_patterns.py에서 쇼츠 패턴 분류 로직을 목업한다.
- /nodes/scene_brief/ 폴더를 생성한다.

- scene_brief.md를 작성하고, create_scene_brief.py에서 장면별 브리프 생성 코드를 작성한다 (GPT 활용).
- /nodes/script_generation/ 폴더를 생성한다.
- script_generation.md를 작성하고, generate_script.py에서 쇼츠 대본 생성 로직을 구현한다.
- /nodes/media_prompt/ 폴더를 생성한다.
- media_prompt.md를 작성하고, create_media_prompts.py에서 이미지/영상 프롬프트를 생성하는 코드를 작성한다.
- /nodes/media_generation_queue/ 폴더를 생성한다.
- media_generation_queue.md를 작성하고, media_queue.py를 작성한다. 추가로 runway_adapter.py, synthesia_adapter.py 파일을 생성하여 API 호출 기능을 구현한다.
- /nodes/media_inspection/ 폴더를 생성한다.
- media_inspection.md를 작성하고, inspect_media.py에서 생성물 품질 체크 로직을 구현한다.
- /nodes/subtitle_cta/ 폴더를 생성한다.
- subtitle_cta.md를 작성하고, generate_subtitles.py와 generate_cta.py를 작성한다 (자막과 CTA/고지 문구 생성).
- /nodes/upload_package/ 폴더를 생성한다.
- upload_package.md를 작성하고, create_upload_package.py에서 최종 영상과 자막 결합 및 설명 작성을 구현한다.
- /nodes/manual_approval/ 폴더를 생성한다.
- manual_approval.md를 작성하여 승인 절차를 문서화한다. UI 연동용 코드는 mock으로 작성한다.
- /nodes/performance_logging/ 폴더를 생성한다.
- performance_logging.md를 작성하고, collect_performance.py에서 성과 API 호출 및 저장 로직을 구현한다 (예: JSON 파일로 저장).
- /nodes/next_experiment/ 폴더를 생성한다.
- next_experiment.md를 작성하고, recommend_next.py에서 성과 데이터를 분석하여 다음 콘텐츠 추천하는 코드를 작성한다.
- 각 노드의 input_schema.json과 output_schema.json에 맞춰 sample_input.json 및 sample_output.json을 준비한다.
- 모든 노드에 대해 테스트용 mock 데이터를 생성하여, Claude Code가 작성한 코드를 검증할 수 있도록 한다.
- 실제 API 키가 없는 경우 mock adapter 또는 예시 데이터를 사용해 먼저 기능을 구현하게 한다.

각 지시사항은 Claude Code에게 전달하여, Antigravity 워크플로의 각 노드에 필요한 코드와 문서를 자동으로 생성하도록 합니다. 이러한 세부 지시를 통해 워크플로의 틀이 갖추어진 후, 실제 API 연동, 에러 처리, 테스트 등을 추가로 개발할 수 있습니다.